

TetrisAi

Jovan Tričković 150/2021

1 Uvod

Pravila igre:

- polje je dimenzija 10×20 ; komadi (tetromini, sedam različitih oblika) padaju jedan po jedan,
- komadi dolaze iz standardnog “7-bag” generatora: svih sedam oblika se nasumično pomeša u torbu, torba se isprazni do praznog, pa se ponovo puni [5],
- pre spuštanja igrač svaki komad može rotirati i pomerati po kolonama; komad pada sve dok ne dodirne pod ili postojeću gomilu, gde se zaključava,
- vrsta popunjena do kraja briše se, a sve vrste iznad nje padaju naniže,
- igra se završava kada se novi komad ne može smestiti na vrhu polja (“top-out”).

Iako pravila deluju jednostavno, Tetris je računarski težak problem. U offline varijanti (ceo niz komada poznat unapred) NP-potpuna je maksimizacija broja očišćenih vrsta, broja postavljenih komada i minimizacija maksimalne visine, uz jaku neaproximativnost prva dva cilja. Igra se prirodno modeluje kao Markovljev proces odlučivanja sa preko 10^{60} konfiguracija polja (2^{200} za prazno/puno), što je čini klasičnim teškim testom za veštačku inteligenciju.

Postoje tri glavna pristupa automatizovanom igranju Tetrisa [6]. Prvi su kontroleri zasnovani na heurističkoj evaluaciji polja. Pionirski kontroler P. Dellacherie-a očišćuje u proseku oko 5 miliona vrsta po partiji, a osmobrajni BCTS skup Thiéry-ja i Scherrera, podešen metodom ukrštene entropije [3, 1, 2], bio je najbolji poznati kontroler svog vremena. Drugi pristup je pretraga (beam ili Monte Carlo) koja planira više poteza unapred nad simuliranim nastavcima. Treći je duboko pojačano učenje. Poznat otvoreni predstavnik prvog pristupa je JavaScript `tetrisai` Y. Lee-a [4].

Ovaj projekat razvija prvi pristup, statičku heurističku evaluaciju polja čije se težine automatski podešavaju genetskim algoritmom (GA), uz numba-JIT kompajliran SRS (Super Rotation System [5]) engine, BFS nabranje svih legalnih mesta sletanja i K -ply beam ($K = 2$) koji dodato raspolaze hold slotom. Sistem je organizovan kao Python paket (`config / tetris_base / search / ga / testing`) sa jednim izvršnim programom (`main.py`) i svim podesivim vrednostima na jednom mestu. Fokusiraćemo se na balans između preživljavanja i skorovanja.

2 Opis rešenja

Arhitektura projekta se sastoji iz četiri stupnja: (1) tablice oblika, SRS rotacija i odskoka (`tetris_base`), (2) engine polja i komada sa numba BFS-om i metrikama polja (`search`), (3) K -ply beam koji bira potez, sa ili bez hold ekspanzije (`search/engine`), i (4) genetski algoritam koji trenira vektor težina (`ga`). Uz njih, paket sadrži i `testing.py` sa preglednikom partije i stres-testom šampiona. Sve vrednosti koje se podešavaju žive u `config.py`. Kompletan pregled konfiguracije dat je u Sekciji 2.8.

2.1 Genetski algoritam

Problem je optimizacija nad 12 realnih težina tako da se maksimizuje složena, diskretna i šumna funkcija cilja (rezultat partije). Funkcija cilja nije diferencijabilna. Već mali pomak težine može da promeni izbor poteza u nekoliko kritičnih trenutaka, što onda nelinearno menja ishod cele partije, pa se metode zasnovane na gradijentu teško primenjuju direktno na samu partiju. Genetski algoritam je izabran jer je bez derivacija, robusan na šum, jednostavan za implementaciju i održava raznolikost populacije, što je korisno jer reljef težina ima mnoge lokalne optimume (npr. težine koje maksimizuju preživljavanje versus težine koje maksimizuju broj tetrisa).

2.2 Kodiranje rešenja i funkcija cilja

Genom je realan vektor $\mathbf{w} \in \mathbb{R}^{12}$. Svaki potez se ocenjuje kao linearni zbir $S = \sum_i w_i f_i$, gde su f_i odlike polja nakon odigranog poteza, opisane u Sekciji 2.3. Beam bira potez (uključujući ev. zamenu sa hold slotom) sa najvećom ocenom duž najbolje linije.

Funkcija cilja (fitness) genoma je

$$\text{fitness} = r_T \cdot P, \quad r_T = 100 \frac{T}{P},$$

gde je T ukupan broj tetrisa, P broj odigranih komada, a r_T stopa tetrisa na 100 komada. Partija je u treningu ograničena na određeni broj komada, radi brzine treniranja. Veće ograničenje daje robustnije rezultate, ali je i trening mnogo sporiji. Budući da se P krati u razlomku i množi spolja, fitness je u konačnom obliku jednostavno $\text{fitness} = 100T$, tj. broj tetrisa skaliran konstantom 100. Ovakav fitness je izabran tako da preživljavanje uvek bude prioritet, ali da se i skor ne ignoriše.

Koristimo samo jedno seme po generaciji, što najbolji genom može da učini malo šumnim. Njegova statistika može biti nešto bolja nego što bi trebao, samo zato što mu se na tom semenu posrećilo. Efekat nije jak jer partije idu do 100k-400k komada, ali može da se desi da genom najbolji po fitnessu u celoj evoluciji nije stvarno najbolji, već samo dovoljno blizu (isti genom na drugom semenu može da umre mnogo ranije). Ovo bi se lako tehnički izbeglo povećanjem broja semena po generaciji, ali i skok sa 1 na 2, 3 ili 4 semena znatno usporava trening. S obzirom na brzinu i mali broj generacija, ovo je prihvatljivo. Dovoljno je izvući i testirati genom koji se već pokazao da preživljava nekoliko generacija kao najbolji. Ostali strateški pokazatelji (lanci tetrisa, dvojni tetrisi, korišćenje hold slot) se prate i beleže po generacijama radi analize.

Svaki genom odigra jednu partiju po jednom semenu generatora komada. Niz komada je identičan za sve genome iste generacije (poređenje niskog šuma), a seme se rotira iz generacije u generaciju ($10\,000 + 100g$ za generaciju g), čime se sprečava prilagođavanje na jedan niz.

2.3 Odlike polja i parametri

Engine izračunava sve odlike u jednom prolazu kroz polje. Kolona 10 (9) je zaštićeni bunar za tetris. Namerno je izuzeta iz tri odlike neravnine (neravnine, prelazi po vrstama i kolonama) jer je njena praznost namerna struktura polja, ali se i dalje računa u visini i rupama, koje ispravno hvataju zaprljani bunar.

- w_0 je vrednost očišćenih linija. w_1 je ukupna visina. w_2 su rupe.
- w_3 do w_5 su neravnine i prelazi (kolona 9 izuzeta).
- w_6 je dubina bunara u koloni 9 (raste samo do dubine 8).
- w_7 su dodatni bunari van kolone 9 (kazna raste trougaono sa dubinom).
- w_8 je visina sletanja. w_9 je zaprljanost bunara. Računa se nakon čišćenja, tako da komad koji upadne u kolonu 9 i odmah očisti vrstu nije kažnjen. Čelije I-komada se ne računaju kao zaprljanost.
- w_{10} je višak visine iznad praga. Ispod praga $H_{\text{knee}} = 12$ vrednost je 0.
- w_{11} je nagrada za duboki bunar (dubina > 4). To je spekulativna odlika koja motiviše zadržavanje bunara dubljeg od jednog tetrisa, što je geometrijski preduslov za doslovni dvojni tetris.

Vrednost očišćenih linija koju množi w_0 je oblikovana skala, umesto sirovog broja linija. Tetris (4 linije) vredi 8, trojka $5 - \pi$, dublka $3 - \pi$ i singl $1 - \pi$, gde je $\pi = 1.5$ kazna za ne-tetris čišćenja. Kazna dakle menja brojeve koje w_0 množi. Efektivne težine po vrsti čišćenja su $w_0 \times \{-0.5, 1.5, 3.5, 8\}$, pa uz pozitivno w_0 singl čišćenje doprinosi negativno, a dublka svega $\frac{1.5}{8}$ vrednosti tetrisa. Birač zato čuva I-komad za tetris umesto da proda bunar za jeftin singl. Uz to, lanac tetrisa unutar pregleda dobija bonus od 4.0 (pola vrednosti tetrisa u skali linija), čime birač voli back-to-back.

Ostali parametri: širina beam-a $B = 5$, dubina $K = 2$, ograničenje partije u treningu na 100 000 komada (finalna provera na punih 400 000), validaciona partija od 2 000 komada po generaciji (na fiksnom semenu 12345, van GA slučajnosti), koja služi za brzu predstavu o trenutnom najboljem genomu. Inicijalizacija težina je ravna, $w_i \sim U(-30, 30)$ za svih dvanaest težina: populacija prve generacije ne zna znak nijedne odlike, pa genetski algoritam sam pronalazi polaritete i kriva učenja kreće od nule.

2.4 BFS pretraga

Projekat koristi BFS pretragu da nađe sve moguće poteze, ali mogu se koristiti i samo finalne pozicije pada (hard drop). Problem sa finalnim pozicijama pada je što se ne mogu isptati sve moguće pozicije. Npr. potezi koji bi se nalazili ispod drugih figura (podmetanje ispod prepusta) ili pozicije do kojih je moguće doći samo putem “wall kick” sistema rotacije ostaju nevidljivi. BFS u svakoj poziciji računa nove pozicije koje se mogu dobiti pomeranjem u desno ili levo ili samom rotacijom (sa svih pet SRS odskoka) i vraća sva legalna stanja mirovanja. Stanje zaključavanja se prihvata samo ako je ceo komad unutar polja, jer je delimično zaključavanje iznad vrha top-out. Ovo je bitno za budući razvoj engine-a koji uči da koristi i T-spinove i za nešto bolji kvalitet poteza generalno. Ipak, mana BFS pretrage je što je vreme treniranja duže nego kod pristupa sa samo finalnim pozicijama pada.

2.5 K-ply pregled, hold i beam pretraga

(Namera je da objašnjenje bude razumljivo i bez poznavanja termina.) U svim verzijama Tetrisa igrač može da vidi barem jedan sledeći tetromino koji mu dolazi, kao i da kešira trenutni komad za kasnije (hold slot). To znači da je standardni vidik 3 moguće različita tetromina, trenutni, sledeći i bankirani. K -ply označava dubinu pregleda, odnosno koliko sledećih figura engine vidi iz next sistema, $K - 1$. Za $K = 2$ vidi samo jednu sledeću figuru, za $K = 3$ dve sledeće itd. Dva sprata su standard, dok je u nekim verzijama Tetrisa taj broj veći. Problem je što za svaku next figuru koju znamo da dolazi moramo iznova da vršimo BFS pretragu, što znači sve sporiji trening što više raste K . Da bismo izbegli prevelika izračunavanja, vršimo beam pretragu. Na svakom spratu uzimamo u obzir samo $B = 5$ najboljih poteza koji se dalje razmatraju, a ostale odbacujemo. Ovo kvira kvalitet konačnog rešenja, ali je neophodno zbog brzine.

Hold slot daje treću dimenziju odluke, ali može da posluži i kao štaka (crutch), jer engine može potencijalno da nauči lošije strategije tokom treninga, ima li hold sistem da ga izbavi. Zato je hold korišćenje podeljeno na dve varijante. Hold-blind dozvoljava hold samo za prvu figuru (jednokratna A/B zamena na nivou poteza). Hold-aware (normalni sistem) dozvoljava beamu da potroši hold na svakom spratu pregleda, pa odigravanje bankiranog komada bankira trenutni (dvostruki hold), pa linija igre u beamu može da zameni komade dva puta zaredom.

2.6 Mehanika genetskog algoritma

Ovde je opisano sve što se tiče GA. Jedna generacija (populacija 24):

1. **Evaluacija.** Svaki genom odigra po jednu partiju na tekućem semenu. Fitnes je opisan u Sekciji 2.2.

2. **Sortiranje** po fitnessu, opadajuće, uz beleženje statistika po genomu.
3. **Elitizam.** Gornjih 10% populacije se prenosi nepromenjeno u sledeću generaciju, čime slučajna mutacija ne može da uništi najbolje nađeno rešenje. Cena je moguća prerana konvergencija, koja se ublažava turnirom umerenog pritiska i oporavkom od stagnacije.
4. **Ukrštanje i mutacija.** Ostatak populacije se popunjava decom.
5. **Oporavak od stagnacije.** Ako najbolji fitness ne poraste za više od 2% tokom $S_{\text{lim}} = 4$ generacija, populacija se popunjava mešavinom potpuno novih nasumičnih genoma i jakih perturbacija šampiona (stopa mutacije 0.5, korak 0.4, promena predznaka 10%), čime se istražuje nova oblast reljefa a da elita nikada nije izgubljena.

Turnirska selekcija. Za svakog roditelja biraju se nasumično $k = 3$ jedinke i uzima se najbolja. Veće k povećava pritisak selekcije (brža konvergencija, manja raznolikost), pa je $k = 3$ umeren, standardan izbor koji balansira konvergenciju i istraživanje. Ruletska selekcija je za ovakav problem obično lošija od turnirske (podložnija preranoj konvergenciji i osetljiva na skaliranje fitnessa), pa koristimo isključivo turnirsku.

Ukrštanje. Implementirana su oba operatora. Uniformno ukrštanje uzima svaku težinu deteta sa verovatnoćom 0.5 od jednog ili drugog roditelja. Jednostavno je i prirodno za realno-vrednosne genome, ali mu je problem što se na dužim treninzima aleli ostaju doslovne kopije vrlo starih jedinki. BLX- α (sa $\alpha = 0.5$) uzorkuje za svaki gen novu vrednost uniformno iz $[\min(p_1, p_2) - \alpha d, \max(p_1, p_2) + \alpha d]$, gde je $d = |p_1 - p_2|$. BLX interpolira i ekstrapolira oko roditelja i time izbegava kopiranje starih alela. Glavni eksperiment rada upoređuje operatore direktno: dva runa sa identičnom početnom populacijom i nizom semena razlikuju se isključivo u operatoru (Sekcija 3).

Mutacija. Za svaki gen, sa verovatnoćom r (stopa mutacije) dodaje se Gausov šum $\mathcal{N}(0, \sigma)$ gde je $\sigma = s \cdot (|w| + s)$, pa veće težine dobijaju veće pomake (proporcionalni drift), a sabirak s drži male težine istraživim. Retko (verovatnoća 2%) se ceo gen promeni predznak, što omogućava istraživanje suprotne polaritete koje Gausov šum ne može sam da dostigne. Stopa i korak linearno opadaju sa napretkom kroz ceo tok, r od 0.30 ka 0.10 i s od 0.30 ka 0.12, što znači rano široko istraživanje i kasno fino podešavanje.

Eval pruning. Radi uštede računanja, u svakoj generaciji prvo igra šampion. Ako šampion odigra celu partiju bez smrti, keširamo njegovu stopu tetrisa. Zatim, kada bilo koji genom pređe 20 000 komada a ima manju stopu od šampionove, njegova se partija prekida i prelazi na sledeću, pod pretpostavkom da genom koji do 20 000 komada nije stigao šampiona, neće ga stići ni kasnije.

2.7 Numba i paralelizam

Genetski algoritmi su veoma povoljni za paralelizaciju, jer je svaki genom nezavisan od ostalih u tekućoj generaciji. Rezultat je povećanje brzine otprilike onoliko puta koliko je logičkih jezgara dodeljeno evoluciji. To zavisi od hardvera, ali je u praksi ubrzanje od 3 do 31 puta. Uz to se koristi i JIT-kompajler numba, pa su sve ključne funkcije (kolizija, metrike, BFS, sagorevanje, oba beama) kompajlirane u mašinski kod i nekoliko puta su brže od čistog Pajtona, sa keširanjem na disk (prvo pokretanje posle izmene kernela košta jednu kompilaciju od nekoliko minuta). Pošto svaki genom može potencijalno da igra do $100k - 400k$ komada, ovakva ubrzanja su neophodna. Svaka partija ima svoje pseudo-slučajno stanje (komad-bag po pozivu), pa paralelne evaluacije ne dele stanje globalnog generatora i determinističke su.

2.8 Kompletna konfiguracija

Sve podesive vrednosti žive u jednom modulu (`config.py`) i ispisuju se na početku izvršne, bez tih podrazumevanih vrednosti.

- engine `hold_aware` ili `hold_blind`,
- ukrštanje `blx` ili `uniform` sa $\alpha = 0.5$,
- prag visine $H_{\text{knee}} = 12$ (numba ga vezuje pri prvom pozivu, pa se postavlja pre uvoza paketa),
- dubina $K = 2$ i širina beam-a $B = 5$,
- kazna za ne-tetris čišćenja 1.5 i bonus lanca 4.0,
- baza semena 10 000 uz rotaciju +100 po generaciji, jedna partija po genomu,
- ograničenje partije u treningu 100 000 komada (finalna provera 400 000),
- prag poboljšanja 1.02 i granica stagnacije 4,
- validaciona partija 2 000,
- populacija 24, generacije 25 (15 na 400k), seme GA 42 (47 za 400k run), veličina turnira $k = 3$,
- broj radnih niti (podrazumevano jezgra -1),
- oznaka izvršne (TAG) za imena izlaznih fajlova u `out/` fascikli.

3 Rezultati

Glavni eksperiment čine dve evolucije: jedna sa BLX- α 0.5 i jedna sa uniformnim ukrštanjem, obe sa konfiguracijom opisanom u Sekciji 2.8 i identičnom početnom populacijom (GA seme 42, ravan prior $U(-30, 30)$). Zatim je pokrenuta treća evolucija, posejana najboljim genomima uniformne evolucije, sa produženim ograničenjem partije na 400 000 komada i trajanjem od 15 generacija, kako bismo dobili genom koji bolje preživljava; ukrštanje je i tu BLX.

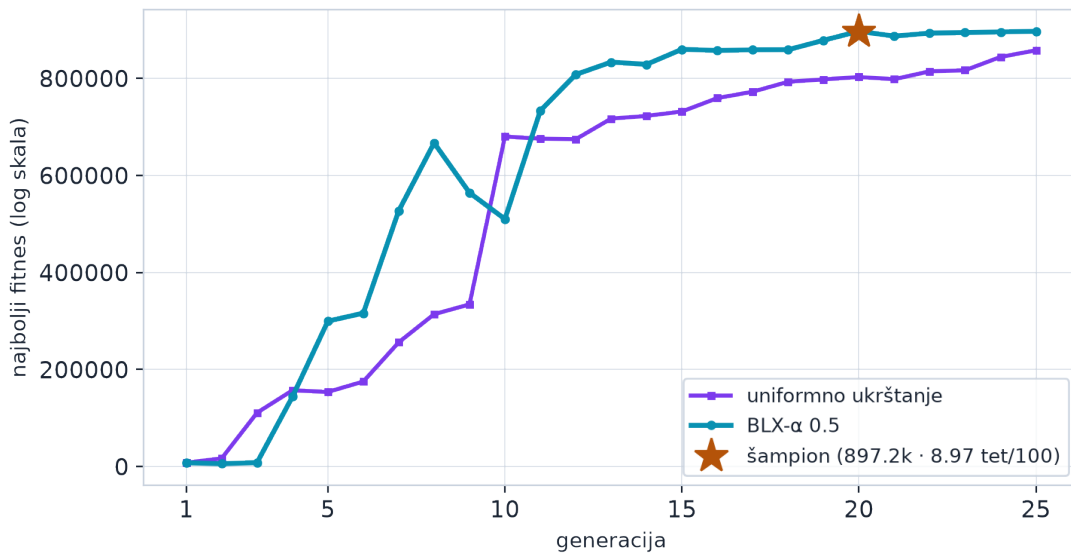


Figure 1: Najbolji fitness po generaciji za dva identično podešena runa. Runovi dele početnu populaciju i niz semena, pa se krive razdvajaju tek od druge generacije. Uniformno ukrštanje (ljubičasto) dostiže 857.9k u 25. generaciji; BLX- α 0.5 (cyan) taj nivo ima već u 15. generaciji i osvaja najboljeg šampiona (zvezdica: 897.2k, tj. 8.97 tet/100).

Uniformni run. Kriva učenja je postepena: fitnes 8k u prvoj generaciji, 100k tek u desetoj i 800k u dvadesetoj. Najbolji genom runa je iz 25. generacije, sa fitnesom 857.9k, tj. stopom od 8.58 tetrisa na 100 komada.

BLX run. Isti recept i isto GA seme, samo BLX- α 0.5 ukrštanje. Napredak je brži: nivo koji uniformno ukrštanje dostiže tek u poslednjoj (25.) generaciji BLX dostiže već u 15. (860.0k), a najbolji genom runa dolazi iz 20. generacije sa fitnesom 897.2k (8.97 tet/100) i težinama

[-0.53, -4.68, -8.06, 10.52, -25.77, -32.38, 33.82, -4.51, -18.06, -11.59,
-35.32, -4.83]

Prednost od +4.6% uz identičnu početnu populaciju i niz semena znači da je razlika posledica samog operatora: uniformno ukrštanje samo bira među postojećim alelima, dok BLX interpolira i ekstrapolira nove vrednosti težina, pa ista populacija pod BLX-om istražuje širi deo prostora pretrage.

Puna dužina partije. Pošto trening teče na partijama od 100k komada, stopa se mora potvrditi i na partijama od 400k. Pokrenut je dakle run sa BLX ukrštanjem i ograničenjem partije na 400k komada, posejan najboljim klasterom uniformnog runa, plus 19 svežih nasumičnih genoma. Najbolji genom runa, iz 11. generacije, preživeo je punih 400k komada sa fitnesom 3.58M, tj. stopom od 8.95 tet/100. Stopa naučena na kraćim partijama se dakle prenosi na punu dužinu partije; duže partije ne menjaju karakter strategije, samo produbljuju dokaz preživljavanja.

Provera na svežim semenima. Finalni šampion BLX runa testiran je na 12 svežih semena (70001 do 70012), po 100k komada, na normalnom hold-aware sistemu. Preživeo je 10 od 12 partija (83%) sa prosekom 8.87 tetrisa na 100 komada; kod preživelih je raspon 8.85 do 8.93. Dve smrti pale su na 5.2k i 76.8k komada, druga pri punoj stopi (8.92) — top-out nije nužno kolaps strategije, ni genom koji igra punom stopom nije imun na nesrećan niz komada. Bezbednost se dakle meri odvojeno od stope i na dovoljno mnogo semena.

Slika 2 prikazuje jednu partiju u pregledniku (`watch_champion`). Svaki potez se animira (komad putuje od spawn-a do mesta sletanja, pa se linije brišu), a bočni panel pokazuje **Hold** i **Next** slotove, brojače komada, linija i kombinacija te kontrolu brzine reprodukcije (ovde 64 $\times$). Preglednik igra aktivni engine iz konfiguracije nad istim genomom, pa služi i kao brza vizuelna kontrola strategije (čuvanje bunara u koloni 9, čekanje I-komada).

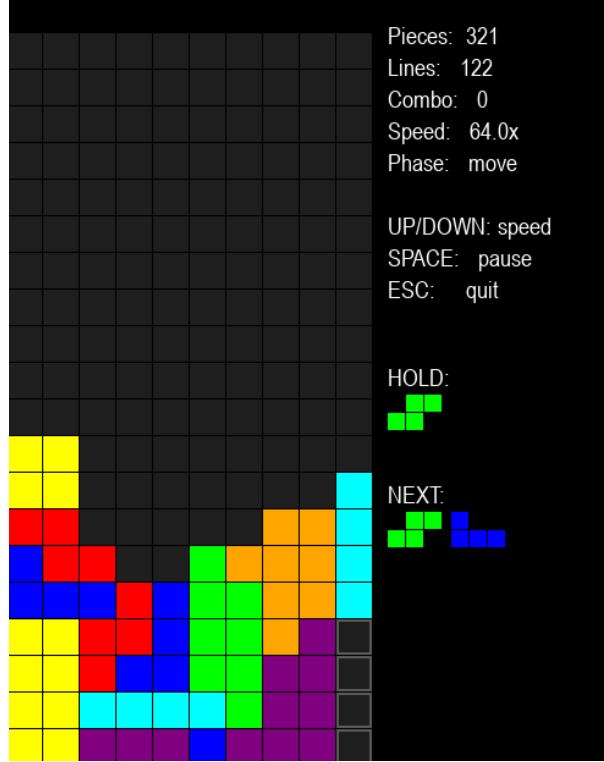


Figure 2: Partija u pregledniku.

Table 1: Pregled runova. Prva dva dele početnu populaciju i niz semena (GA seme 42, ravan prior $U(-30, 30)$, partije od 100k, 25 generacija) i razlikuju se samo u operatoru ukrštanja; treći proverava stopu na punoj dužini (partije od 400k, 15 generacija, start iz najboljeg klastera uniformnog runa). Nezavisna provera: 12 svežih semena po 100k komada, hold-aware sistem.

	uniformni run	BLX run	BLX 400k
engine	hold_blind	hold_blind	hold_blind
ukrštanje	uniformno	BLX- α 0.5	BLX- α 0.5
ograničenje partije	100k	100k	400k
start	od nule	od nule	iz klastera
najbolji fitnes	857.9k	897.2k	3.58M
stopa (tet/100)	8.58	8.97	8.95
generacija najboljeg	25	20	11
provera 12×100k	—	10/12 @ 8.87	12/12 @ 8.91

Zapažanja iz razvoja, korisna za tumačenje rezultata. (a) Najdominantnija pozitivna težina šampiona je dubina bunara ($w_6 = 33.8$), a najteže kazne su višak visine iznad praga ($w_{10} = -35.3$) i zaprljanost bunara ($w_9 = -18.1$); rupe ($w_2 = -8.1$) su kod ovih genoma blaže kažnjene od zaprljanosti bunara, u duhu strategije prvo ne izgubi. (b) Strmo w_{10} deluje kao sigurnosni ventil koji motiviše hitno čišćenje pre top-out-a. (c) Statistika po generacijama (lanci, dvojni tetrise, i-hold) pokazuje da velika većina tetrise dolazi u lancima (na validaciji šampiona 151 od 182), pa strategija ne troši I-komade pojedinačno. (d) Kill bar i champ-first mehanizam, uz partije od 100k, omogućili su da populacija 24 kroz 25 generacija prođe za oko dva sata po runu na običnom procesoru.

Poređenje sa literaturom. Merilo u literaturi je prosečan broj očišćenih linija po partiji do kraja igre. Originalni Dellacherie-ov kontroler dostiže oko 5 miliona linija, a CEM-podešeni

osmobrojni kontroler Thiéry-ja i Scherrer-a oko 35 miliona [1, 2], a klasifikacioni pristup CBMPI i 51 milion [6]. Naš cilj balansira skor i preživljavanje, pa brojevi nisu direktno uporedivi. Naše preživljavanje je samo donje ograničeno, jer je partija ograničena na 400k komada. Dodatni problem su i implementacioni detalji: bez zajedničkog simulatora, sigurno poređenje nije moguće.

4 Zaključak

Rad razvija dobro poznatu liniju Tetris AI-ja zasnovanih na odlikama: SRS engine, BFS nabranje svih legalnih mesta sletanja, K -ply beam sa hold slatom i genetsko podešavanje težina. Glavni zaključci su:

1. Pri plitkoj pretrazi ($K = 2$) strategija mora da bude izražena kroz statičke odlike, a genetski algoritam tada samo fino podešava magnitude unutar ručno oblikovanog prostora. Dubina pregleda je deo strategije, pa se genom treniran na $K = 2$ ne prenosi na $K = 4$.
2. Operator ukrštanja sam menja ishod: uz identičnu početnu populaciju i niz semena, BLX- α 0.5 daje 8.97 prema 8.58 tet/100 uniformnog ukrštanja, a nivo finalnog šampiona dostiže deset generacija ranije. Mehanizam je interpolacija i ekstrapolacija alela naspram prepakivanja kopija osnivača. Fitnes je u svim runovima ista veličina (broj tetrisa), pa su rezultati direktno uporedivi.
3. Visoka stopa tetrisa nije automatski dobar šampion. Finalni šampion sa stopom 8.97 tet/100 preživeo je 10 od 12 svežih semena (83%), a jedna smrt je pala pri punoj stopi (8.92 tet/100 u trenutku top-out-a). Bezbednost se meri odvojeno od stope i na dovoljno mnogo semena.
4. Lanac tetrisa (back-to-back) je izvodljiv i čini većinu ostvarenih tetrisa. Doslovni dvojni tetris je geometrijski redak (bunar dubine 8) i u tenziji sa preživljavanjem.

Pravci daljeg unapređenja: (a) uslovljena nagrada za duboki bunar (samo kada je I-komad vidljiv u pregledu), čime bi se izbeglo slepo zadržavanje bunara i smanjila stopa top-out-a, (b) veća dubina pregleda ($K \geq 3$) uz veću populaciju, pošto je prenos strategije kroz dubinu ograničen, (c) T-spin podrška, koju BFS nabranje mesta već omogućava, (d) potencijalno korišćenje CEM-a ili neuronskih mreža i poređenje rezultata sa GA-om.

Literatura

References

- [1] C. Thiéry, B. Scherrer. *Building Controllers for Tetris*. ICGA Journal 32(1), 2009, str. 3 do 11. Otvoreni pristup: <https://inria.hal.science/inria-00418954>.
- [2] C. Thiéry, B. Scherrer. *Improvements on Learning Tetris with Cross Entropy*. ICGA Journal 32(1), 2009, str. 23 do 33. Otvoreni pristup: <https://inria.hal.science/inria-00418930>.
- [3] I. Szita, A. Lőrincz. *Learning Tetris Using the Noisy Cross-Entropy Method*. Neural Computation 18(12), 2006, str. 2936 do 2941.
- [4] Y. Lee. *tetrisai*, Tetris AI napisan u JavaScriptu (4 odlike, iscrpni 2-ply pregled, genetsko podešavanje težina). github.com/LeeYiyuan/tetrisai, 2014 do 2017.
- [5] Tetris Wiki. *Super Rotation System, Random Generator*. <https://tetris.wiki/>, pristupljeno septembra 2026.

- [6] S. Algorta, "O. Şimşek. *The Game of Tetris in Machine Learning*. arXiv:1905.01652, 2019.
<https://arxiv.org/abs/1905.01652>.